

Malicious model deployed in HF repo to reversed RCE and worm infection by RagRetriever.from_pretrained() with complete bypass of pickle scanning in [huggingface/transformers](#)

✓ Valid

Reported on Dec 5th 2023

Description

The Hugging Face (HF) repo enables the remote users to fetch RagRetriever models from the hub by simply running the official demo function code `RagRetriever.from_pretrained()`. However, this demo function parses the remote config.json at first and can be misled by `"index_name"` and `"index_path"` to implicitly redirect the victim users to load pickle files from the other repos specified by `"index_path"`, hence suffering deserialization attacks. In this vulnerable condition, the attackers have two repos, one is a frontend one to phish the victims to run the demo code `RagRetriever.from_pretrained(frontend)`, and the other is a backend repo that holds the real attacking pickle file. The frontend repo redirects the victim users to load the backend one by setting `{"index_path": backend}` in its config.json. As the victim users are likely not aware of the existence of the backend repo, they just find the frontend repo safe enough, meaning the complete bypass of HF' pickle scanning.

Proof of Concept

The following steps show how to exploit unsafe pickle.load from RagRetriever models for reversed RCE and even worm infection over huggingface platform (We have a demo video deployed at <https://zpbrent.github.io/pocs/demo-RagRetriever-worm.rar> with password `huggingfaceraag` for these steps):

- 1), we have an attacker huggingface account `zpbrent` and a victim one `pzhoutest`
- 2), the attacker `zpbrent` deploys a RagRetriever model `zpbrent/RagRetriever` for phishing. Note that the huggingface performs a pickle scanning to alert `unsafe file` for malicious pickle files, such as the backend repo deployed at `zpbrent/RagReuse`. However, our phishing repo `zpbrent/RagRetriever` can completely bypass this scanning.
- 3), the victim `pzhoutest` do not have the RagRetriever model `pzhoutest/RagRetriever` before being attacked, and also no `HACKED` and `HACKED-2` files in the disk.
- 4), for reversed RCE, the pre-requisite is the victim `pzhoutest` has been attracted by the pretrained model deployed at repo `zpbrent/RagRetriever`.

4.1), s/he finds this repo nothing "unsafe" and thus downloads this pretrained model using huggingface's official `from_pretrained` function

```
RagRetriever.from_pretrained("zpbrent/RagRetriever").
```

4.1), the victim observes the model downloading complete.

4.3), but meanwhile the `HACKED` and `HACKED-2` files have been illegally created in the disk (meaning the success of reversed RCE).

5), for worm infection, an additional pre-requisite is the victim `pzhoutest` has readily logged in huggingface with a "write" permission (`huggingface-cli login --token xxx`) when running the official `from_pretrained` function for downloading.

5.1), a new repo `pzhoutest/RagRetriever` has been implicitly created and uploaded in victim `pzhoutest` 's account, with the same contents of `zpbrent/RagRetriever` (meaning the success of worm infection).

PoC repos:

Frontend (public upon request):

<https://huggingface.co/zpbrent/RagRetriever>

Backend:

<https://huggingface.co/zpbrent/RagReuse>

Impact

This vulnerability can be exploited to execute arbitrary codes and OS commands in the victim machine that runs the demo function `RagRetriever.from_pretrained()` remotely, leading to reversed RCE and even worm infection. We have three new insights for this one:

1, we can completely bypass the pickle scanning and make all the pickle files stored at the frontend phishing repo whitelisted.

2, we take a step further to show how to exploit this kind of vulnerability to infect pickle malware across HF.

3, this report involves two deserialization vulnerabilities from `RagRetriever`, one is triggered by

```
psgs_w100.tsv.pkl
```

 and the other is by

```
hf_bert_base.hnswSQ8_correct_phi_128.c_index.index_meta.dpr.
```

⚙️ We are processing your report and will contact the **huggingface/transformers** team within 24 hours. 2 years ago



✎ Peng Zhou modified the report 2 years ago



✎ Peng Zhou modified the report 2 years ago



Peng Zhou modified the report 2 years ago



Peng Zhou modified the report 2 years ago



Peng Zhou modified the report 2 years ago

We have contacted a member of the **huggingface/transformers** team and are waiting to hear back 2 years ago



A **huggingface/transformers** maintainer validated this vulnerability 2 years ago

Thanks for this report - our open source team implemented a solution that will require users to set an env. variable (TRUST_REMOTE_CODE) in order to access pickle.load in transformers code. Full details on the PR can be found at <https://github.com/huggingface/transformers/pull/27776>. After deeper review, please note this fix also addresses <https://huntr.com/bounties/e1a3e548-e53a-48df-b708-9ee62140963c/>.

\$ **zpbrent** has been awarded the disclosure bounty

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



A **huggingface/transformers** maintainer marked this as fixed in **4.36** with commit **1d63b0** 2 years ago

\$ The fix bounty has been dropped



This vulnerability has now been published 2 years ago

[Sign in](#) to join this conversation

CVE

CVE-2023-6730

Published

Vulnerability Type

CWE-502: Deserialization of Untrusted Data

Severity

Critical (9)

Attack vector

Network

Attack complexity

Low

Privileges required

Low

User interaction

Required

Scope	Changed
Confidentiality	High
Integrity	High
Availability	High

[Open in visual CVSS calculator](#) 

Registry

Pypi

Affected Version

v4.35.2

Visibility

Public

Status

Fixed

Disclosure Bounty

\$1500

Fix Bounty

\$375

Found by



Peng Zhou

@zpbrent

LIGHTWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.

